KOENIG

step forward



Kotlin Syntax and Idioms

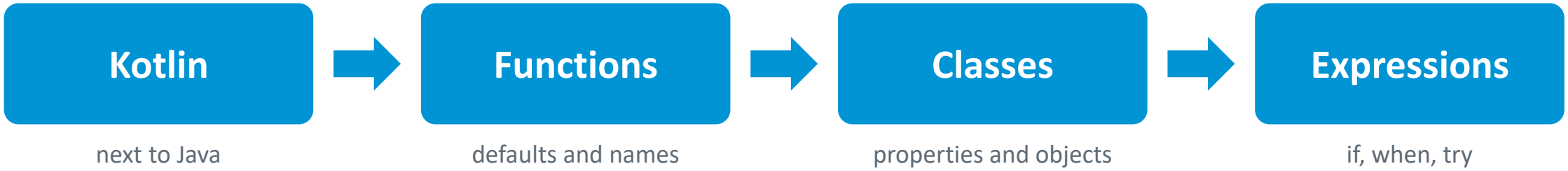
Kotlin for Java Developers

What You Will Learn

1. Kotlin on the JVM, compiler and IntelliJ setup
2. val versus var, type inference, basic types
3. Functions: default, named, single-expression
4. Classes, constructors, properties, visibility, inheritance
5. Objects, companion objects, top-level functions
6. if, when, try, loops, String templates, multiline strings.

Demo Scenario: ShopKart in Kotlin

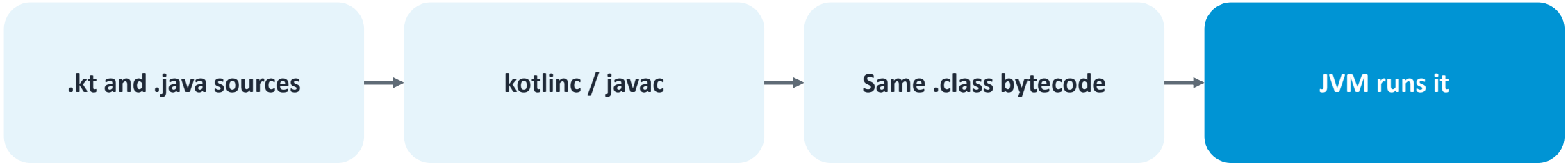
ShopKart's model and a small shipping add-on are Java. We add Kotlin next to it, one file at a time, and write the shipping code the way a Kotlin developer would.



We will cut

- **Boilerplate**
- **Overloads**
- **Static helpers**

Kotlin on the JVM



- **Same platform:** Kotlin compiles to JVM bytecode, runs on JDK 21 like Java.
- **100% interop:** call Java from Kotlin and Kotlin from Java.
- **Same tools:** Gradle, IntelliJ, JUnit, Spring Boot, your libraries.
- **Statically typed** like Java: errors are found at compile time.

Compiler, Standard Library, Build

Compiler

kotlinc turns .kt files into .class files. IntelliJ and Gradle call it for you.

Standard library

kotlin-stdlib: List, String helpers, let, also, ranges. A project library.

Build tool

IntelliJ builds Kotlin and Java together. Gradle: plugin org.jetbrains.kotlin.jvm.

- **We use** Kotlin 2.3.10, the compiler bundled with IntelliJ.
- **Java 21** for both languages.

Java is not replaced: Java and Kotlin share one module

IntelliJ Project Setup for Kotlin

```
File > New > Project > Kotlin
    Name:          ShopKartKotlin
    Build system:  IntelliJ
    JDK:           21

src/
    setup/  Main.kt          // Kotlin file

Run: green arrow next to main     // runs MainKt
```

- **Kotlin compiler:** bundled with IntelliJ, nothing to download.
- **Standard library:** one project library, added once.

No build file: Ctrl+F9 builds Kotlin and Java together

Mixed Java and Kotlin Projects

Project layout

```
src/
    model/SampleData.java              // Java
    shipping/Shipment.java             // Java
    shipping/ShipmentService.java      // Java
    setup/SetupMain.kt                 // Kotlin
    classes/Parcel.kt                  // Kotlin
    pojo/ShipmentData.kt               // Kotlin
```

- **Both directions:** Kotlin sees Java classes, Java sees Kotlin classes.
- **Getters/setters** of Java classes look like properties in Kotlin.
- **Migrate gradually:** one class at a time, tests stay green.

```kotlin
// Kotlin using the Java Shipment class
val s = Shipment(1, "Laptop", "", ShipStatus.PACKED, 3, "Electronics")
s.status = ShipStatus.SHIPPED   // calls setStatus
```

Demo: First Kotlin File in a Java Project

What we build

- **SetupMain.kt:** a top-level main.
- **SampleData:** Java, called from Kotlin.
- **shipment.status:** a setter as a property.

How the concept helps

- **Same project:** Java and Kotlin together.
- **No rewrite:** migrate file by file.
- **Same JVM:** same tools.

val versus var

Java

```java
final String name = "Ada";
int count = 0;
count++;

// name = "Bob";   // compile error
```

Kotlin

```kotlin
val name = "Ada"
var count = 0
count++

// name = "Bob"   // compile error
```

- **val:** read-only reference, like final. Prefer it.
- **var:** can be reassigned. Use only when needed.
- **Read-only is not immutable:** a val list can still change its content.

Rule: start with val, change to var only if the compiler asks

Type Inference

Java

```java
String title = "Report";   // Java 10+: var
var size = 3;
List<String> names =
    new ArrayList<>();
```

Kotlin

```kotlin
val title = "Report"          // String
val size = 3                  // Int
val names = mutableListOf<String>()
val ratio: Double = 3.0       // explicit
```

- **The compiler** infers the type from the right-hand side.
- **Static typing stays:** `val size = 3` can never become a String.
- **Write the type** for public functions, or when it helps the reader.

Basic Types

Int
32 bit, like int e.g. 42, 1000

Long
64 bit, like long e.g. 42L

Double
like double e.g. 2.5

Boolean
true / false e.g. true

Char
single character e.g. 'K'

String
immutable text e.g. "Hi"

```
val a: Int = 7
val b: Long = a.toLong()   // no implicit widening
val c = 7 / 2              // 3 (Int division)
val d = 7 / 2.0            // 3.5
```

- **No primitives** in source code; the compiler uses int, long ... when possible.

Demo: val, var and Types

What we build

- **val:** cannot be reassigned.
- **Inference:** Int, Double, String.
- **toLong():** no silent widening.

How the concept helps

- **Compiler:** guards the intent.
- **Fewer vars:** fewer bugs.
- **Conversions:** always visible.

Functions

Java

```java
public static int add(int a, int b) {
    return a + b;
}

public static void greet(String name) {
    System.out.println("Hi " + name);
}
```

Kotlin

```kotlin
fun add(a: Int, b: Int): Int {
    return a + b
}

fun greet(name: String) {
    println("Hi $name")
}
```

- **fun** keyword; type after the name: `a: Int`.
- **No void:** no return value means Unit, which you can omit.
- **No semicolons.** No class needed for a function.

Default Arguments

Java

```
String row(String name, int price) {
    return row(name, price, 12);
}
String row(String name, int price, int width) {
    // ... real work
}
```

Kotlin

```
fun row(
    name: String,
    price: Int,
    width: Int = 12
): String {
    // ... real work
}
```

- **One function** replaces a family of overloads.
- **Defaults** are evaluated at call time and can use earlier parameters.

No more telescoping constructors and overload chains

Named Arguments

```
row("Laptop", 65000)                    // all defaults
row("Mouse", 799, 8)                    // width only
row("Pen", 20, rupee = true)            // skip width
row(price = 60, name = "Notebook")      // any order
createUser("Ada", true, false, 3)       // unclear
createUser("Ada", admin = true, active = false, retries = 3)
```

- **name = value** makes calls self-documenting, especially for Boolean flags.
- **Skip defaults** you do not care about; order does not matter once named.
- **Once you name one,** name the rest after it (unless positions are unchanged).

Single-Expression Functions

Java

```
int gst(int price) {
    return price * 18 / 100;
}

String stockLabel(int qty) {
    return qty > 0
        ? "In stock" : "Sold out";
}
```

Kotlin

```
fun gst(price: Int) = price * 18 / 100

fun stockLabel(qty: Int) =
    if (qty > 0) "In stock"
    else "Sold out"
```

- **= expression** replaces { return ... }; the return type is inferred.
- **Best for** one-liners: mappers, getters, small helpers.
- **Use a block** when the body needs several statements.

Demo: Defaults, Names and Expression Bodies

What we build

- **row():** width has a default.
- **Named:** rupee = true.
- **gst():** one expression.

How the concept helps

- **One function:** no overload family.
- **Clear calls:** names, any order.
- **Short:** type inferred.

Classes and Primary Constructors

Java

```java
public class Parcel {
    private final long id;
    private String label;

    public Parcel(long id, String label) {
        this.id = id;
        this.label = label;
    }
    // + getters and setters ...
}
```

Kotlin

```kotlin
class Parcel(
    val id: Long,
    var label: String
)

val p = Parcel(1, "Laptop for Asha")
// no new keyword
```

- **Primary constructor** sits in the class header.
- **val / var** before a parameter creates the property as well.
- **Classes are final** by default; write `open` to allow inheritance.

Properties

```
class Parcel(val id: Long, var label: String, priority: Int = 3) {

    var priority: Int = priority.coerceIn(1, 5)
        set(value) { field = value.coerceIn(1, 5) } // setter

    val isDelivered: Boolean                    // computed
        get() = status == ShipStatus.DELIVERED

    var status: ShipStatus = ShipStatus.PACKED
}

parcel.label = "New label"        // calls setLabel(...)
println(parcel.label)             // calls getLabel()
```

val

getter only (read-only property).

var

getter and setter.

field

backing field inside custom accessors.

Secondary Constructors and init

```
class Parcel(val id: Long, var label: String) {

    init {                          // runs with primary
        require(label.isNotBlank()) { "label must not be blank" }
    }

    constructor(label: String) : this(0, label)
}

val a = Parcel(5, "Laptop")
val b = Parcel("Quick parcel")
```

- **init** blocks run in order, as part of the primary constructor.
- **require()** throws IllegalArgumentException.
- **constructor(...)** secondary; delegates with `: this(...)`.
- **Prefer defaults** over secondary constructors when you can.

Visibility Modifiers

public
default; visible everywhere

internal
same module only

protected
class and subclasses (not the package)

private
same class or file (top level)

```
class Parcel {
    var note: String = ""
        private set     // read: all, write: inside

    internal fun debugInfo() = "parcel " + id  // module only
}
```

- **No package-private:** internal is module-wide.
- **Default is public,** unlike Java's package default.

Demo: Parcel, a Kotlin Class

What we build

- **Constructor:** val id, var label.
- **Setter:** priority from 1 to 5.
- **private set:** note is read-only outside.

How the concept helps

- **No getters:** properties instead.
- **Validation:** in init and setters.
- **Visibility:** public by default.

Inheritance: Final by Default

Java

```java
public class Payment {
    public String label() { ... }
}

public class UpiPayment extends Payment {
    @Override public String label() {
        return "UPI: " + super.label();
    }
}
```

Kotlin

```kotlin
open class Payment(val amount: Int) {
    open fun label() = "Payment of Rs " + amount
}

class UpiPayment(amount: Int) : Payment(amount) {
    override fun label() = "UPI: " + super.label()
}
```

- **Final by default:** write open to allow extension.
- **Members too:** open fun, then override fun.
- **The colon:** replaces extends and the super call.

Interfaces and Abstract Classes

Java

```java
interface Shippable {
    String shippingLabel();
    default int fee() { return 0; }
}

abstract class Courier {
    abstract int cost(int km);
}
```

Kotlin

```kotlin
interface Shippable {
    fun shippingLabel(): String
    fun fee() = 0               // default body
}
abstract class Courier {
    abstract fun cost(km: Int): Int
}
```

- **interface:** members may have bodies, no default keyword.
- **A class** extends one class, implements many interfaces.
- **abstract:** no body, class cannot be created.
- **override** is mandatory in the subclass.

Demo: Payment, Shippable and Courier

What we build

- **Payment:** final until it is open.
- **UpiPayment:** override and super.
- **Shippable, Courier:** interface and abstract.

How the concept helps

- **Safe default:** no accidental subclass.
- **Explicit:** open and override.
- **Compiler:** abstract cannot be created.

Objects: Singletons

Java

```java
public final class IdGenerator {
    private static long last = 0;

    private IdGenerator() { }

    public static long next() {
        return ++last;
    }
}
// IdGenerator.next()
```

Kotlin

```kotlin
object IdGenerator {
    private var last = 0L

    fun next(): Long = ++last
}

// IdGenerator.next()
```

- **object** declares a class and creates its single instance, thread-safe on first use.
- **Use for** stateless helpers, registries, counters.

Companion Objects

Java

```java
public class Voucher {
    public static final int MAX_PERCENT = 50;
    public static Voucher welcome() {
        return new Voucher("WELCOME10", 10);
    }
}
// Voucher.welcome()
```

Kotlin

```kotlin
class Voucher private constructor(
    val code: String, val percent: Int
) {
    companion object {
        const val MAX_PERCENT = 50
        fun welcome() = Voucher("WELCOME10", 10)
    }
}
// Voucher.welcome()
```

- **Kotlin has no static:** members of the companion object are called through the class name.
- **const val** for compile-time constants; factory functions for named creation.

Top-Level Functions

Java

```java
public final class PercentUtils {
    private PercentUtils() { }

    public static boolean isValidPercent(int p) {
        return p >= 1 && p <= 50;
    }
}
// PercentUtils.isValidPercent(60)
```

Kotlin

```kotlin
// file ShopKartUtils.kt
const val SHOP_NAME = "ShopKart"

fun isValidPercent(p: Int) = p in 1..50

// isValidPercent(60)
```

- **No utility class** needed: functions and constants live directly in the package.
- **Java callers** see a class ShopKartUtilsKt with static methods.
- **main()** is a top-level function too, so SetupMain.kt becomes SetupMainKt.

Demo: object, companion and Top-Level

What we build

- **IdGenerator:** an object.
- **Voucher:** companion factories.
- **ShopKartUtils.kt:** a top-level function.

How the concept helps

- **No static:** three better homes.
- **Named factories:** Voucher.welcome().
- **No Utils class:** functions in the package.

Expressions Over Statements

Statement: does something

```
String label;
if (done) {
    label = "Finished";
} else {
    label = "Open";
}
```

Expression: produces a value

```
val label = if (done) "Finished" else "Open"
val code = when (state) { ... }
val n = try { text.toInt() } catch (e: Exception) { 0 }
```

- **In Kotlin** if, when, try and throw all have a value.
- **Result:** fewer mutable variables, fewer null states.

if as an Expression

Java

```
int max = a > b ? a : b;

String badge;
if (parcel.isDelivered()) {
    badge = "[x]";
} else {
    badge = "[ ]";
}
```

Kotlin

```
val max = if (a > b) a else b

val badge =
    if (parcel.isDelivered) "[x]" else "[ ]"

// no ternary operator in Kotlin
```

- **else is required** when if is used as a value.
- **Block form:** the last expression in the braces is the result.
- **No `?:` ternary;** Kotlin's `?:` is the elvis operator

when as an Expression

Java

```
switch (status) {
    case PACKED:
        return "Waiting";
    case SHIPPED:
        return "On the way";
    case DELIVERED:
        return "At the customer";
    default:
        return "Unknown";
}
```

Kotlin

```
fun statusLabel(status: ShipStatus) = when (status) {
    ShipStatus.PACKED -> "Waiting"
    ShipStatus.SHIPPED -> "On the way"
    ShipStatus.DELIVERED -> "At the customer"
}
// no break, no default needed
// compiler checks every case
```

- **No fall-through,** no break.
- **Exhaustive:** all enum entries listed means no else; a new entry breaks compilation.
- **More than switch:** works with ranges, types and conditions (next slide).

when Beyond switch

```
fun priorityLabel(p: Int) = when (p) {
    5 -> "CRITICAL"
    4 -> "HIGH"
    3 -> "NORMAL"
    1, 2 -> "LOW"           // several values
    else -> "INVALID"       // required: when is an expression
}

fun band(p: Int) = when {   // no subject: plain conditions
    p !in 1..5 -> "INVALID" // range check
    p >= 4 -> "ACT NOW"
    else -> "LATER"
}

fun describe(x: Any) = when (x) {
    is String -> "text of ${x.length} chars"   // type check + smart cast
    is Int -> "number"
    else -> "other"
}
```

Replaces if-else chains

Equal values, ranges, conditions.

Replaces instanceof

is checks and smart casts the variable.

Branch with a block

Use braces when a branch needs more lines.

try as an Expression

Java

```
int priority;
try {
    priority = Integer.parseInt(text);
} catch (NumberFormatException e) {
    priority = 3;
}
```

Kotlin

```
val priority = try {
    text.trim().toInt()
} catch (e: NumberFormatException) {
    3
}

val title = input ?: throw
    IllegalArgumentException("required")
```

- **Value:** the last expression of the try block, or of the catch block that ran.
- **throw** is an expression too, so it fits on the right of `?:`.
- **No checked exceptions** in Kotlin: catch only what you can handle.

Demo: if, when and try as Expressions

What we build

- **if:** gives the badge.
- **when:** over ShipStatus, exhaustive.
- **try:** parsePriority.

How the concept helps

- **Fewer variables:** no assign later.
- **Compiler:** checks every case.
- **One construct:** values, ranges, types.

Loops and Ranges

Java

```
for (int i = 1; i <= 3; i++) { ... }
for (int i = 1; i < 3; i++) { ... }
for (int i = 10; i >= 0; i -= 5) { ... }

while (stock > 0) { stock--; }
```

Kotlin

```
for (i in 1..3) { ... }                 // 1, 2, 3
for (i in 1 until 3) { ... }            // 1, 2
for (i in 10 downTo 0 step 5) { ... }  // 10, 5, 0

while (stock > 0) { stock-- }
```

- **A range** 1..3 includes the end, until excludes it.
- **downTo and step** count down and skip.
- **No counter** variable to declare or get wrong.
- **while** is unchanged; for walks any list too.

String Templates

Java

```
String line = "#" + s.getId() + " "
    + s.getLabel() + " ["
    + s.getStatus() + "]";

String pct = String.format(
    "%d of %d (%.1f%%)", d, n, p);
```

Kotlin

```
val line =
    "#${s.id} ${s.label} [${s.status}]"

val name = "Ada"
println("Hi $name, ${name.length} chars")

val pct = "$d of $n (${"%.1f".format(p)}%)"
```

- **$name** inserts a variable.
- **${expression}** inserts any expression: calls, arithmetic, if.
- **To print a dollar sign:** \$ or ${'$'}.

Multiline Strings

Java

```
String card = "Parcel:   " + title + "\n"
            + "Status:   " + status + "\n"
            + "Priority: " + prio;

String json = "{\"id\": " + id
    + ", \"label\": \"" + title + "\"}";
```

Kotlin

```
val card = """
    Parcel:   $title
    Status:   $status
    Priority: $prio
""".trimIndent()

val json = """{"id": $id, "label": "$title"}"""

val help = """
    |Usage:
    |  list
""".trimMargin()
```

- **Triple quotes:** raw string, quotes and line breaks need no escapes; templates still work.
- **trimIndent():** removes the common indent. trimMargin(): cuts up to the | character.

Demo: Templates and Raw Strings

What we build

- **Template:** $name and ${...}.
- **Format:** a percentage in the text.
- **Raw string:** trimIndent, trimMargin.

How the concept helps

- **Reads like output:** no plus signs.
- **No escapes:** inside triple quotes.
- **Same value:** less noise.

Key Takeaways

1 Same JVM, same tools
The Kotlin compiler in IntelliJ; Java and Kotlin mix freely.

2 val first
val and var with inference; no implicit widening.

3 Functions
Default and named arguments, single-expression bodies.

4 Expressions
if, when and try produce values; when is exhaustive.

5 Strings
Templates and raw strings replace concatenation.

6 Classes
Properties, object, companion, top-level functions.



Happy Learning :)